SYSTEM ARCHITECTURE & HIGH CONCURRENCY DESIGN

สถานการณ์ที่ 3: การออกแบบ Architecture และ Logic ป้องกัน Database ล่มและ Overbooking

> **โจทย์และสถานการณ์ที่ 3**
>
> บริษัทจัดแคมเปญแจกคูปอง 100 สิทธิ์ โดยมีผู้ใช้กดเข้ามาพร้อมกันหลักหมื่นคนในเวลา 12.00 น. คุณจะออกแบบ Architecture และ Logic การทำงานอย่างไรเพื่อไม่ให้ Database ล่มและป้องกันการแจกคูปองเกิน (Overbooking)?

คำตอบและแนวทางการออกแบบ (Architecture & Logic)

ห้ามให้ Request ทั้งหมื่นคนวิ่งไปชน Relational Database ตรงๆ เด็ดขาด ใช้ **In-Memory Data Store (เช่น Redis)** เป็นด่านหน้ารับแรงกระแทกและตัดโควตาแบบ Atomic ก่อน แล้วส่งเฉพาะผู้โชคดี 100 คนไปลง Database แบบ Asynchronous โดยมีด่านหน้ากรอง Bot และจำกัดการกดรัว (Rate Limit) มีด่านตัดสิทธิ์ใช้ Atomic Operation ตัดคูปอง 100 ใบในหน่วยความจำ คนที่ 101 เป็นต้นไปจะถูกดีดกลับทันทีโดยไม่แตะ Database และสุดท้าย ด่านบันทึกข้อมูล ส่งรายชื่อผู้ที่ได้สิทธิ์ 100 คนเข้า Queue เพื่อให้ Worker ค่อยๆ บันทึกลง Database

โครงสร้างด่านการทำงาน (3-Tier Protection Architecture)

ด่านการทำงาน	เครื่องมือ / กลไกหลัก	หน้าที่และการทำงาน
1. ด่านหน้า	Bot Protection & Rate Limit	กรอง Bot และจำกัดการกดรัวของผู้ใช้งานก่อนเข้าสู่ระบบหลังบ้าน
2. ด่านตัดสิทธิ์	In-Memory Data Store (Redis Atomic Operation)	ตัดคูปอง 100 ใบในหน่วยความจำแบบ Atomic คนที่ 101 เป็นต้นไปจะถูกดีดกลับทันทีโดยไม่แตะ Database
3. ด่านบันทึกข้อมูล	Message Queue (RabbitMQ) & Worker	ส่งรายชื่อผู้ที่ได้สิทธิ์ 100 คนเข้า Queue เพื่อให้ Worker ค่อยๆ บันทึกลง Database แบบ Asynchronous

รายละเอียด Logic การทำงานเพื่อป้องกัน Overbooking และ Database ล่ม

- **Logic การตัดสิทธิ์แบบ Atomic บน Redis (ป้องกัน Overbooking):** ห้ามทำ `SELECT` แล้วค่อย `UPDATE` แต่ต้องรวบเป็นคำสั่งเดียวที่ทำงานแบบ Single-threaded Atomic บน Redis เช่น เตรียมข้อมูลก่อนเวลา 12:00 น. ให้ตั้งค่าใน Redis ไว้ล่วงหน้า: `coupon:101:quota = 100` และ `coupon:101:users = Set ว่างๆ` เมื่อผู้ใช้กดรับคูปอง ให้ API เรียกหลังบ้านตัดสิทธิ์บน Redis
- **การป้องกัน Database ล่มด้วย Message Queue คั่นกลาง:** เมื่อคัดผู้โชคดี 100 คนจาก Redis ได้แล้ว เราไม่ควรยิง `INSERT` ลง Database ตรงๆ ในจังหวะที่ระบบกำลังพีค ควรใช้ Message Queue คั่นกลาง
- **ขั้นตอนการตอบกลับและบันทึกข้อมูล (Asynchronous Processing):** เมื่อ API ได้รับค่า `1` จาก Redis ให้ส่ง Message เข้าไปที่ Message Queue (RabbitMQ) แล้วตอบกลับหน้าบ้านทันทีว่า *"คุณได้รับสิทธิ์แล้ว กำลังจัดส่งคูปองเข้ากระเป๋า"* จากนั้นให้ Worker ดึงข้อมูลจาก Queue มา `INSERT` ลงตาราง `user_coupons` ใน Database ด้วยความเร็วที่ DB รับไหว